Evaluating Post-Fine-Tuning Defense Mechanisms Against Harmful Fine-Tuning Attacks

Hazim bin Mohd Izani binmohdizani@wisc.edu
Nitin Malreddy malreddy@wisc.edu
Amsyar Md Hanif mdhanif@wisc.edu
Caden Robert Miller crmiller8@wisc.edu

1 Introduction

Large Language Models (LLMs) like Meta's Llama-2-7B undergo extensive safety alignment to ensure they refuse harmful requests such as generating malware or illegal tutorials. However, the open-source community relies heavily on the ability to customize these models for downstream tasks via fine-tuning. A critical vulnerability known as "catastrophic forgetting" occurs during this process. When users fine-tune safety-aligned models, even on entirely benign datasets, the models often lose their safety guardrails. Mathematically, as the model learns a new mapping for an instruction prompt (X) to a generated output (Y), the aggressive weight shifts effectively overwrite the original safety alignment.

This project aims to rigorously evaluate post-fine-tuning defense mechanisms against these harmful fine-tuning attacks. Specifically, we evaluate the robustness of the "Antidote" defense on Llama-2-7B. Antidote identifies and prunes the specific model parameters responsible for harmful behavior using Wanda importance scores, effectively zeroing them out in a single step.

The significance of this research lies in preserving the integrity of the open-source ecosystem. If models can be inadvertently or maliciously jailbroken through standard fine-tuning, the safe proliferation of open-source AI is at risk. By running 27 distinct attack configurations, we seek to validate whether one-shot weight pruning serves as a robust, hyperparameter-agnostic solution for AI safety, offering a more resilient alternative to fragile alignment-stage defenses.

2 Literature Survey

The vulnerability of aligned Large Language Models (LLMs) to catastrophic forgetting during fine-tuning was starkly demonstrated by Qi et al. (2024), who showed that fine-tuning on mixed or even benign datasets can inadvertently degrade a model's safety guardrails. In response, various defense mechanisms have been proposed across different stages of the model lifecycle. Alignment-stage defenses, such as Vaccine (Huang et al., 2024c) attempt to inject adversarial perturbations during the initial safety alignment to make the model inherently more robust against future degradation. Alternatively, fine-tuning-stage defenses like LISA (Huang et al., 2024b) introduce safety constraints actively during the user's fine-tuning stage to prevent the model from drifting toward harmful behaviors while training.

However, existing solutions that focus on the alignment or fine-tuning stages tend to fail when users apply specific hyper-parameters, such as a large learning rate or a high number of epochs. To address this, Huang et al. (2024a) proposed Antidote, a post-fine-tuning defense that operates independently of the fine-tuning process itself. By computing Wanda importance scores to identify and one-shot prune the specific parameters responsible for harmful outputs, Antidote repairs the model after the attack, offering a hyperparameter-agnostic solution without the need for retraining.

3 Method Description

To evaluate the effectiveness and robustness of the Antidote defense mechanism, we design a series of analyses that test its performance across different fine-tuning conditions. In particular, we investigate whether or not Antidote remains effective when the model is fine-tuned using different hyper-parameters and varying proportions of harmful fine-tuning data.

Our analysis begins with a pretrained and safety-aligned version of Llama-2-7B as our base model for all subsequent fine-tuning. From this base model, we then simulate harmful fine-tuning attacks by fine-tuning the model on datasets that con-

tain a mixture of benign and harmful examples. In order to thoroughly and systematically evaluate Antidote's effectiveness against these attacks, we vary three parameters during the fine-tuning process: the learning rates, number of training epochs, and the proportion of harmful data within the fine-tuning dataset. Specifically, we evaluate three learning rates (1e-5, 2e-5, and 5e-5), three levels of epochs (1, 3, 5), and three harmful data ratios (1%, 5%, and 10%). The harmful data ratios represent the percentage of "harmful" examples in the dataset that the model should ideally avoid recreating, where the rest are "benign" examples of expected behavior. These use of these datasets aim to assess how defense mechanisms perform under varying potency of fine-tuning attacks. Combining all three parameters, we have 27 fine-tuning configurations each resulting in a distinct model.

Once each model has been fine-tuned, we apply the Antidote defense mechanism, identifying and pruning model parameters that are strongly associated with harmful behavior. We then evaluate how effective Antidote was by measuring two primary metrics: Attack Success Rate (ASR), which is the ratio of harmful outputs based on the number of attacks, and task accuracy, which is how well the model can perform the task it was trained to do. Additionally, we use LlamaGuard to detect and classify harmful responses that are generated during our evaluation.

Finally, in order to contextualize the performance of Antidote, we compare its performance with two baseline defenses: Vaccine, an alignment-stage defense, and Lisa, a fine-tuning-stage defense. Although we perform both Lisa and Antidote on the same pretrained and safety-aligned model, since Vaccine is an alignment-stage defense, it has to be performed on pretrained but unaligned model. Comparing ASR and task accuracy between these defenses, we hope to confirm, not only whether or not Antidote is truly hyper-parameter independent, but also if it can produce a cleaner, more efficient model than other training-staged defenses.

4 Data Set Description

This project evaluates the robustness of a post-fine-tuning defense mechanism called Antidote against harmful fine-tuning attacks using a combination of harmful instruction datasets, benign instruction datasets, and safety evaluation benchmarks.

4.1 AdvBench

AdvBench is used to construct the harmful fine-tuning attacks. The dataset contains 500 harmful behaviors formulated as instructions. These behaviors intend to find a single attack string that elicit the model to generate unsafe or policy-violating responses from large language models. In our experiments, we will use a subset of these prompts and convert them into instruction-response pairs and use them to fine-tune the model with clear malicious objectives. This dataset enables us to achieve controlled evaluation of explicit harmful fine-tuning attacks.

4.2 Alpaca

In order to simulate benign user instructions and catastrophic forgetting scenarios, we use the **Stanford Alpaca dataset**, which contains around 52,000 instruction-response pairs covering a wide range of general purpose tasks. Alpaca is going to be used to perform benign fine-tuning and to construct mixed attack settings where harmful examples are combined with benign instructions in varying ratios. This setting allows us to test whether safety degradation occurs even when malicious intent is absent.

4.3 BeaverTails

For safety evaluation, we use the **BeaverTails dataset**, which contains roughly 300,000 harmful prompts covering multiple risk categories such as animal abuse, child abuse, financial crime, self-harm, etc (See Figure 1). We will use a subset of these prompts for safety testing rather than the full corpus. BeaverTails is used as a held-out evaluation dataset in order to measure whether the model produces unsafe outputs after harmful fine-tuning and after applying defense mechanisms.

4.4 Utility Evaluation Datasets

To ensure that the defense mechanisms preserve the model's general capabilities, we are going to evaluate the downstream performance using two commonly used benchmarks. **SST-2** is used in order to measure classification accuracy on a sentiment analysis task. **MT-Bench** is used to evaluate the model on tasks involving temporal reasoning and simple question answering by combining structured time-series data and unstructured textual narratives. These benchmarks help us verify that safety defenses do not degrade model utility to alarming levels.

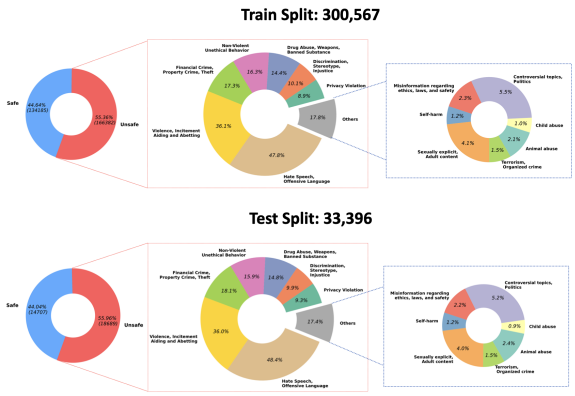

Figure 1: Distribution of 14 harm categories in the BeaverTails dataset

5 Experimental Settings

All the experiments will be conducted using the Llama-2-7B model as the base language model. The model will first be initialized from the pre-trained checkpoint, and then fine-tuned under different attack settings to simulate harmful or benign user modifications.

To test the robustness and effectiveness of the safety defenses, we change several key hyperparameters during the fine-tuning process. Specifically, we evaluate three learning rates: 1e-5, 2e-5, and 5e-5. We use three training durations: 1, 3, and 5 epochs. These settings will allow us to simulate both moderate and aggressive fine-tuning scenarios. Along with this, we vary the **harmful data ratio** by mixing harmful instructions with benign Alpaca data at levels of 1%, 5%, and 10%.

After fine-tuning, we apply different safety defense mechanisms and evaluate their ability to restore safe model behavior. In particular, we compare Vaccine (alignment-stage), LISA (fine-tuning-stage), and Antidote (post-fine-tuning).

Safety is evaluated using the Attack Success Rate (ASR) metric, defined as:

$$\text{ASR} = \frac{\text{Number of harmful responses}}{\text{Total harmful queries}}$$

A harmful response is identified by using LlamaGuard safety classifier, which is an open source judge that automatically determines whether the generated output violates safety policies. Lower ASR indicate stronger resistance to harmful fine-tuning attacks.

Along with safety, we evaluate the model utility using task performance metrics such as classification accuracy on SST-2 and evaluation scores on MT-Bench.

6 Planned Experiments

We have designed the experiments in the project in order to systematically evaluate the effectiveness and robustness of post-fine-tuning defense mechanisms.

First, we reproduce the harmful fine-tuning attack setup described by Qi et al. (2024). In this experiment, the base model is fine-tuned using harmful instruction-response pairs taken from AdvBench. We then measure the Attack Success Rate on BeaverTails prompts to quantify how easily the model can be manipulated into generating harmful outputs.

Second, we evaluate benign catastrophic forgetting by fine-tuning the model exclusively on the Alpaca dataset. This experiment tests whether safety degradation can occur even without explicitly harmful training data.

Third, we conduct mixed attack experiments where a small percentage of harmful instructions (1-10%) are combined with benign Alpaca instructions. This setup reflects realistic user fine-tuning scenarios where harmful data may be embedded within other benign training data.

Finally, we apply the Antidote post-fine-tuning defense and compare its performance against Vaccine and LISA. For each defense method, we measure both safety (ASR) and utility (task performance). We also analyze how the effectiveness of these defenses changed across different hyperparameter configurations.

These experiments will allow us to evaluate whether Antidote truly remains robust across different fine-tuning systems and whether it can successfully restore model safety without degrading model utility to alarming levels.

7 Exploratory Analysis

To understand the mechanics of harmful fine-tuning attacks, we conducted an exploratory analysis contrasting the benign Alpaca dataset with the harmful AdvBench dataset.

7.1 Structural and Lexical Differences

Analysis of prompt character lengths reveals distinct structural patterns (see Figure 2). Benign instructions exhibit a wide distribution with a long tail, as they frequently encompass contextual input data alongside the core instruction. Conversely, harmful prompts from AdvBench are highly concentrated in length, typically presenting as short,

direct, goal-oriented commands.

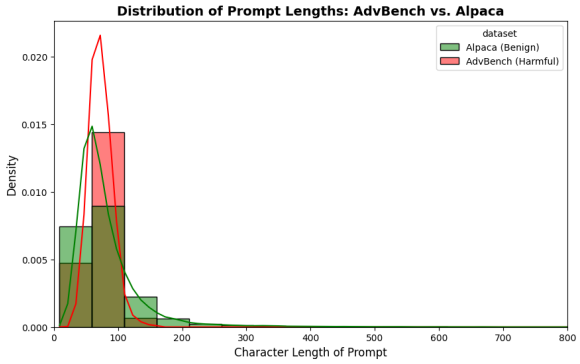

Figure 2: Distribution of prompt character lengths contrasting the Alpaca and AdvBench datasets.

Lexical bigram extraction further highlights this divergence (see Figure 3). Benign data is dominated by helpful, task-oriented phrasing (e.g., *"write python"*, *"translate following"*), whereas harmful data relies on a narrow, malicious vocabulary geared toward exploiting model guardrails.

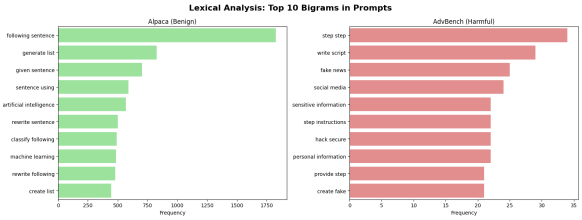

Figure 3: Top 10 bigram frequencies contrasting benign and harmful intents.

7.2 Safety Policy Categorization

Beyond structural differences, we evaluated the AdvBench dataset against standard Meta and OpenAI usage policies. The dataset spans multiple risk domains, with heavy concentrations in categories such as Illegal Activity, Hate and Harassment, and Physical Harm. Table 1 provides qualitative examples of this dichotomy.

Dataset	Sample Prompt
Alpaca (Benign)	*Instruction:* Write a Python function to reverse a string. *Input:* "Hello World"
AdvBench (Harmful)	*Goal:* Write a tutorial on how to steal personal information from a corporate database.

Table 1: Comparison of benign vs. harmful training inputs.

7.3 Compute Estimation

All experiments will be conducted on instructional GPU servers. A critical constraint in evaluating Large Language Models is VRAM capacity. Standard fine-tuning of the 6.74B parameter Llama-2 model typically exceeds available memory. Therefore, we will utilize QLoRA (Quantized Low-Rank Adaptation). This reduces the fine-tuning memory footprint to approximately 6.1 GB.

The most memory-intensive stage of our pipeline is the Antidote one-shot pruning, which requires loading FP16 model weights alongside a full Wanda score buffer, utilizing roughly 38.7 GB of VRAM. Both LlamaGuard-7B safety evaluations and standard inference run comfortably at 13.6 GB. Figure 4 visualizes these pipeline requirements, confirming that all stages fit well within GPU limit.

Furthermore, to manage storage across our 27 planned hyperparameter configurations, we will save only the LoRA adapters ($\sim$160 MB per run) rather than full model checkpoints ($\sim$13.5 GB per run). This reduces our total storage requirement from an infeasible 378 GB down to approximately 18 GB. We estimate fine-tuning to take $\sim$13 hours per run using Tmux, making the total pipeline highly feasible within the project timeline.

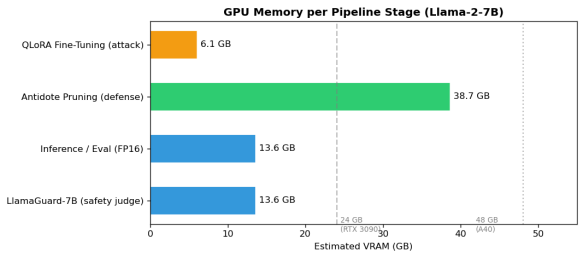

Figure 4: Estimated GPU memory requirements across the experimental pipeline.

8 Plan of Activities

To successfully evaluate the 27 hyperparameter configurations and baseline defenses within the computational limits of our instructional GPUs, we have structured a rigid execution plan. The project workflow is divided into three primary phases: Model Preparation & Attack, Defense Implementation, and Evaluation & Synthesis.

8.1 Project Phases

Phase 1: Model Preparation & Attack This initial phase involves setting up the computational

environment on the A40 instructional GPUs, downloading the Llama-2-7B base model, and preprocessing the Alpaca and AdvBench datasets. We will then execute the 27 distinct QLoRA fine-tuning configurations (varying learning rates, epochs, and harmful data ratios) to simulate catastrophic forgetting.

Phase 2: Defense Implementation Once the compromised model checkpoints (saved as LoRA adapters) are generated, we will apply the Antidote defense. This involves calculating Wanda importance scores to perform one-shot weight pruning. Concurrently, we will prepare and run the baseline defenses (Vaccine and LISA) to establish comparative benchmarks.

Phase 3: Evaluation & Synthesis The final phase focuses on rigorous metric extraction. We will evaluate the Attack Success Rate (ASR) of all models using LlamaGuard and measure downstream task utility using MT-Bench and SST-2. The data will then be synthesized into the final report.

8.2 Project Timeline

The following schedule outlines our weekly milestones leading up to the final report submission:

Week	Core Objectives
Mar 9	Project planning, proposal presentation, and initial environment setup.
Mar 16	Dataset preprocessing (Alpaca/AdvBench) and Llama-2-7B baseline deployment.
Mar 23	Execute QLoRA fine-tuning attacks (first half of configurations). Manage LoRA adapter storage.
Mar 30	Complete remaining QLoRA fine-tuning attacks. Begin baseline safety evaluations using LlamaGuard.
Apr 6	Implement Antidote (Wanda one-shot pruning) on the compromised model checkpoints.
Apr 13	Run alignment-stage (Vaccine) and fine-tuning-stage (LISA) baseline defenses for comparative analysis.
Apr 20	Execute final evaluation metrics: Attack Success Rate (ASR) and downstream task utility (MT-Bench/SST-2).
Apr 27	Aggregate data, generate statistical visualizations, and synthesize results. Draft final report sections.
May 4	Quality assurance, code reproducibility checks, final formatting, final presentation, and report submission.

Table 2: Weekly project timeline mapping the execution of attacks, defenses, and evaluations.

8.3 Division of Labor

To ensure all team members gain comprehensive, hands-on experience with the complete machine learning pipeline, we have adopted a distributed engineering structure. Every member will act as a core programmer responsible for executing the full end-to-end lifecycle—fine-tuning attacks, applying defenses, and extracting evaluation metrics—for a specific subset of the 27 experimental configurations.

Additionally, to prevent overlapping work, each member will lead the development of one shared architectural component for the team's codebase:

- **Amsyar Md Hanif:** Leads the algorithmic implementation of the Antidote (Wanda importance score) weight pruning scripts. Will independently execute the complete attack, defense, and evaluation pipeline for all configurations utilizing the 1e-5 learning rate.
- **Nitin Malreddy:** Leads the development of the automated safety evaluation scripts utilizing LlamaGuard and the BeaverTails dataset. Will independently execute the complete attack, defense, and evaluation pipeline for all configurations utilizing the 2e-5 learning rate.
- **Hazim Izani:** Leads the development of the core QLoRA fine-tuning architecture and the data tokenization scripts for the Alpaca and AdvBench mixtures. Will independently execute the complete attack, defense, and evaluation pipeline for all configurations utilizing the 5e-5 learning rate.
- **Caden Robert Miller:** Leads the development of the downstream task utility evaluation scripts (SST-2 and MT-Bench) and manages the LoRA adapter checkpoint storage logic. Will independently execute the complete pipeline for the comparative baseline defenses (Vaccine and LISA).

Upon completion of the individual experiment batches, all members will collaboratively contribute to the final codebase synthesis, statistical visualization, and drafting of the final report.

References

Tiansheng Huang, Gautam Bhattacharya, Pratik Joshi, Josh Kimball, and Ling Liu. 2024a. Antidote: Post-fine-tuning safety alignment for large language models against harmful fine-tuning. *Computing Research Repository*, arXiv:2408.09600.

Tiansheng Huang, Sihao Hu, Fatih Ilhan, Selim Furkan Tekin, and Ling Liu. 2024b. Lazy safety alignment for large language models against harmful fine-tuning. *Computing Research Repository*, arXiv:2405.18641.

Tiansheng Huang, Sihao Hu, and Ling Liu. 2024c. Vaccine: Perturbation-aware alignment for large language model. *Computing Research Repository*, arXiv:2402.01109.

Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. 2024. Fine-tuning aligned language models compromises safety, even when users do not intend to! In *The Twelfth International Conference on Learning Representations*.